AeroMind

Team
Jihad ElSayed, Brooks Adams, Jesse Holt, Nick, Emmad
Date
2/23/2026

Executive Summary

Aim for ¾ to one whole page but no more. If your project is more research in nature…if you wish to try to submit a publication out of the project, please write the summary more as an abstract of a research paper. Otherwise, write the summary as a technical executive summary. If you have questions about this, please email me.

If you need some tips…
http://www.umuc.edu/writingcenter/writingresources/exec_summaries.cfm

AeroMind is a research-driven software system that investigates the reliability, robustness, and safety of real-time hand-gesture control for a consumer drone (DJI Tello TLW004). While gesture-based control offers an intuitive human–machine interface, it introduces variability due to differences in users, environmental conditions, and network latency. Most hobby drone systems prioritize autonomy or basic remote control and do not systematically evaluate gesture robustness under controlled experimental conditions. This project addresses that gap through a structured implementation and empirical evaluation framework.

The system consists of a real-time perception and control service that performs static hand-gesture recognition using a webcam, maps gestures to discrete drone commands, and enforces safety constraints through confidence thresholds, temporal stability windows, command cooldown logic, and a high-priority emergency stop. Commands are transmitted over Wi-Fi using the Tello SDK, and all predictions, decisions, and timestamps are logged for quantitative analysis. A JavaFX-based operator interface provides live visualization of gesture predictions, system state, and safety decisions.

Controlled experiments will be conducted with multiple participants under varying lighting, background, and distance conditions. Performance will be evaluated using accuracy, precision, recall, confusion matrices, end-to-end latency, false command rate, and emergency stop success rate.

Rather than assuming gesture control is reliable, AeroMind defines the measurable operational limits under which it remains safe and effective. The project contributes a modular, safety-aware architecture and an experimentally grounded evaluation of human–drone interaction under realistic offboard control constraints.

Table of Contents

Once you have created your document using the appropriate headings, you can create your table of contents automatically using Word's table of contents.

Statement of the Problem

What is the problem, and why are you trying to solve it? Give a background on the domain area. What is currently being done in the area?

Objectives

What are you going to do?

We will design, implement, and experimentally evaluate a real-time hand-gesture control system for a consumer drone (DJI Tello TLW004). The system performs gesture recognition using a webcam, maps gestures to discrete drone commands, enforces safety constraints through a confidence threshold and stability window, and transmits commands over Wi-Fi using the Tello SDK { Tello SDK 2.0 User Guide.pdf} .

We will conduct controlled experiments with multiple participants under varying lighting, background, and distance conditions 04_experiment_plan.md . All predictions, commands, and timestamps will be logged for quantitative evaluation 06_methodology.md .

How will your proposed project contribute to the domain you chose?

The domain is **human–drone interaction and offboard robotic control**.

Most hobby drone systems:

- Focus on autonomy
- Ignore human variability
- Do not systematically evaluate safety under environmental variation

Our contribution is:

1. A modular, safety-aware gesture control architecture.
2. A controlled experimental evaluation of gesture robustness across users and environmental conditions

02_research_questions_hypotheses

.

3. Quantitative analysis using defined accuracy, latency, and safety metrics

05_metrics_definition

.

4. A reproducible logging framework enabling empirical analysis instead of anecdotal claims.

This moves the discussion from "gesture control works" to **"gesture control works under these measurable constraints."**

How will it solve the problem?

The core problem is that gesture-based drone control can be unreliable and unsafe due to user variability, environmental conditions, and network latency

01_problem_statement.md

The project addresses this through engineered safety mechanisms and empirical evaluation. Confidence thresholds, temporal stability windows, command cooldown logic, and a high-priority emergency stop reduce false or unstable commands. All interactions are logged and analyzed using defined accuracy, latency, and safety metrics

05_metrics_definition.md

Rather than assuming reliability, the system experimentally determines under which conditions gesture control remains safe and effective, thereby defining its operational limits.

Technical Approach

I don't expect this to be "ironed out" right now, but please provide as much thought and detail here as you can. You must include a UI, it will be written in Java. You can use JavaFX if you would like to make the UI easier to build.

1) High-level architecture

Split the system into **two processes**:

A) Python "Perception + Control Service" (real-time)

- Captures webcam frames
- Runs hand landmark detection + gesture classification
- Applies stability + safety rules
- Outputs "current gesture", "confidence", "candidate command", "decision allowed/blocked + reason"
- Sends Tello commands over UDP (or simulator)
- Logs everything to CSV for experiments
  This already matches our current modules: mapper + safety + drone interface + logger

B) JavaFX UI (operator console) ("A previous Python prototype exists; however, the final interface will be implemented using JavaFX.")

- Shows live video preview (either a stream from Python or a simple MJPEG feed)
- Shows real-time prediction, confidence, stability timer, safety block reason
- Shows drone telemetry (battery, height)
- Provides operator controls:
  - Start/Stop system
  - Toggle SIM/DRONE
  - Emergency stop button (always available)
  - Change thresholds (confidence, stable window, cooldown)
  - Participant + condition fields (lighting/background/distance)
  - Start/stop recording runs
- Displays "Safety Status" clearly (green allowed, red blocked, why)

This is exactly the UI we need for demo + research runs.

2) Process-to-process communication (simple and robust)

Use **local TCP or WebSocket** between JavaFX and Python.

Python → JavaFX (telemetry + inference stream)
Send JSON messages at ~10–30 Hz:

```
{
  "ts_ms": 1700000000000,
```

```
  "pred_gesture": "forward",
  "confidence": 0.91,
  "stable_ms": 520,
  "candidate_cmd": "forward 20",
  "decision_allowed": true,
  "block_reason": "none",
  "battery_pct": 87,
  "height_cm": 45,
  "mode": "drone"
}
```

JavaFX → Python (commands/config)

- `SET_PARAMS {conf:0.8, stable_ms:400, cooldown_ms:900}`
- `SET_METADATA {participant:"P2", lighting:"low", background:"cluttered", distance_m:1.0}`
- `MODE sim|drone`
- `EMERGENCY`
- `START_RUN`, `STOP_RUN`

Why this is good:

- Your control loop stays real-time in Python (where OpenCV/MediaPipe belongs)
- JavaFX stays clean and responsive
- You can test UI without a drone connected

3) Video in the JavaFX UI (practical options)

Pick one:

Option A (easiest): MJPEG/HTTP stream from Python

- Python serves a `/video` endpoint streaming MJPEG frames
- JavaFX loads it (or you decode frames yourself)

Option B: send compressed frames over the same socket

- Python JPEG-encodes frames
- JavaFX decodes and renders into an `ImageView`

Either way, the UI shows what the model sees, which is huge for debugging and demos.

4) Safety + control logic (what actually prevents stupid behavior)

Keep these policies (already in your code):

- **Emergency stop overrides everything**

safety

- Block commands if:
    - confidence < threshold
    - stable_ms < stable_window_ms
    - within cooldown since last command

safety

- When blocked, the UI must show **why** (low confidence vs not stable vs cooldown)

Also align with Tello behavior:

- Use SDK mode + UDP commands + state telemetry

5) Experiment mode (research-grade, not vibes)

JavaFX UI is the "experiment console":

- Set participant + conditions (lighting/background/distance)

04_experiment_plan

- Start a run → Python generates run_id and logs to CSV
- UI shows live counters:
    - command suppression rate
    - false command events (if you tag ground truth in UI)
    - emergency stop activations

You already have the logging schema in place

main

— the UI just fills metadata cleanly so your CSV is usable.

6) Testing plan (so it doesn't collapse on demo day)

Unit tests (fast)

- GestureMapper stability timing gesture_mapper.md
- SafetyLayer decision rules (threshold, stable, cooldown, emergency) safety.md
- Command formatting and mapping correctness

Integration tests

- JavaFX ↔ Python messaging (JSON parse, reconnect, dropped packets)
- SIM mode end-to-end: gesture → decision → simulator update

simulator

System tests

- Drone connected:
  - "command" handshake
  - send takeoff, land, emergency
  - telemetry reads battery/height
- Latency measurement from frame timestamp to command dispatch (log it)

7) What you'll deliver (clear and grade-friendly)

- JavaFX UI app (runs independently)
- Python service (runs sim or drone)
- Logged CSV dataset for participants/conditions
- Metrics report: accuracy, confusion matrix, latency, false command rate, emergency stop success 05_metrics_definition.md

ct Schedule

A Gantt chart here would be really nice, but you should have some sort of laid out plan for how you will delegate responsibilities. Remember to adhere to the lifecycle you chose when you planning this out, and remember to coordinate drone time with your partnering team.

Project Schedule and Responsibility Delegation

The AeroMind project follows an iterative incremental lifecycle. The system will first be implemented and validated in simulator mode before drone integration. After achieving baseline stability, the team will conduct controlled drone experiments and finalize performance evaluation.

Lifecycle Phases

Phase 1 – Requirements and Baseline Implementation (Weeks 1–2)

- Finalize system architecture and communication design.
- Implement gesture recognition baseline.
- Implement gesture mapping and safety logic.

- Integrate structured logging.
- Validate full pipeline in simulator mode.

Lead: Jihad
Support: Emmad

Phase 2 – UI Integration and System Stabilization (Weeks 3–4)

- Develop JavaFX operator console.
- Implement real-time telemetry display.
- Integrate UI with perception/control service.
- Tune confidence threshold and stability parameters.
- Perform extended simulator testing.

Lead (UI): Nick
Lead (Integration): Jihad
Support: Emmad

Phase 3 – Drone Integration and Validation (Weeks 5–6)

- Connect to DJI Tello via SDK.
- Validate command transmission and telemetry feedback.
- Perform controlled safety tests (emergency stop, command suppression).
- Tune parameters under real network latency conditions.

Drone sessions will be scheduled in coordination with the partnering team. Simulator validation must pass before each drone session to avoid wasted flight time.

Lead: Jihad
Safety & Setup: Emmad
Observation & Documentation: Brooks, Jesse

Phase 4 – Experimental Data Collection (Week 7)

- Conduct structured trials with participants.
- Test across lighting, background, and distance conditions.
- Log predictions, confidence, latency, and command outcomes.
- Ensure consistent experimental protocol across all participants.

Drone time will be pre-booked in blocks and limited to validated experimental runs only.

Lead (Research): Brooks
Lead (Protocol & Analysis): Jesse
System Operator: Jihad
UI Monitoring: Nick

Phase 5 – Analysis and Reporting (Week 8)

- Compute accuracy, precision, recall, confusion matrices.
- Measure end-to-end latency and safety metrics.
- Interpret condition-based performance differences.
- Finalize report and prepare demo presentation.

Lead: Brooks & Jesse
Technical Validation: Jihad

Drone Time Coordination Plan

To ensure efficient use of the drone:

- No drone testing occurs without simulator stability.
- Emergency stop and safety gating must be verified before flight.
- Drone sessions are scheduled in advance with the partnering team.
- Each session has a predefined goal (connectivity test, safety validation, or official data collection).
- If system instability occurs, testing reverts immediately to simulator mode.

This plan ensures:

- Clear delegation of responsibilities.
- Proper adherence to the iterative lifecycle.
- Safe and efficient coordination of limited drone time.
- Balanced workload between engineering and research roles.

Deliverables

What you promise to deliver here is how I will grade your final product. If you promise A, B, and C, but you can only do A and B, you will need to justify why C was not possible. You may also consider the approach of promising A, and if time permits, you will attempt B and/or C. As long as A is substantial enough, I do not mind this approach.

A. Fully Functional Gesture-Control System (Primary Deliverable)

- Real-time static gesture recognition.
- Gesture-to-command mapping with safety mechanisms (confidence threshold, stability window, cooldown, emergency stop).
- Simulator mode for safe validation.
- Drone integration using the DJI Tello SDK.
- JavaFX-based operator UI displaying live prediction, confidence, safety status, and telemetry.
- Structured logging of all trials for analysis.

This is the guaranteed deliverable and represents a complete, substantial system.

B. Experimental Evaluation and Metrics (If Baseline Is Stable)

- Controlled multi-participant testing.
- Accuracy, precision, recall, and confusion matrix analysis.
- End-to-end latency measurement.
- Safety metric evaluation (false command rate, emergency stop success).

C. Extended Enhancements (Time Permitting)

- Advanced visualization of metrics in the UI.
- Additional robustness testing under extreme conditions.
- Parameter tuning interface for live experimentation.

Budget

If your project would require a real budget if put into production, please conduct the research and provide that budget. If you would want additional sensors, how much would they cost? Would a more high-powered drone be better? If so, what would be the cost? How many person hours will it require to build this project out? The textbook has information on estimating project costs. Please refer ot that.

The current project uses a DJI Tello TLW004 drone (~$100–$150) and a standard laptop with webcam, resulting in minimal hardware cost for the prototype.

If deployed in production, estimated costs would include:

- Higher-powered drone (e.g., DJI Mini series): $400–$800
- Dedicated high-resolution camera: $100–$300
- Onboard computing unit (e.g., NVIDIA Jetson): $300–$700
- Additional sensors (depth camera or LiDAR for safety): $200–$800

Estimated production-grade hardware cost: $1,000–$2,500 per unit, depending on configuration.

Development effort is estimated at approximately 250–350 total person-hours, including design, implementation, testing, drone validation, experimentation, and reporting.

The current prototype remains low-cost by leveraging existing equipment and open-source software, while the production estimate reflects improved reliability, sensing, and processing capability.

Special Topics

Optional section. May or may not be necessary depending on project. Mention any additional details that you feel appropriate.

This project emphasizes safety-first design in human–drone interaction. Because drone control involves physical motion and potential risk, the system prioritizes command suppression, stability checks, and an emergency stop override over aggressive responsiveness.

The architecture is intentionally modular, separating perception, safety logic, UI, and drone communication. This allows future extensions such as dynamic gestures, onboard processing, multi-drone coordination, or integration with autonomous navigation.

The system also supports reproducible experimentation through structured logging and controlled environmental variation, enabling future research on bias, robustness, and human variability in gesture-based control systems.

Team Qualifications

List out each person's strengths and role on the project.

Jihad ElSayed
Strengths: System architecture, integration, real-time control logic, drone communication.
Role: System Architect and Integration Lead; responsible for perception pipeline, safety logic, and Tello SDK integration.

Brooks Adams
Strengths: Research design, experimental planning, data interpretation.
Role: Research Lead; responsible for experimental protocol, metrics analysis, and results documentation.

Jesse Holt
Strengths: Statistical reasoning, structured analysis, documentation.
Role: Experimental Analysis Lead; responsible for participant coordination, data evaluation, and report writing.

Nick
Strengths: UI development, frontend structure, visualization.
Role: JavaFX UI Lead; responsible for operator console, real-time display, and user interaction components.

Emmad
Strengths: Testing, validation, support tasks.
Role: Support and Testing Engineer; responsible for simulator testing, safety validation, and drone session setup.